HTTP Methods in FastAPI

This topic is extremely important because HTTP methods are the **foundation of APIs**.
Whenever:

- a frontend talks to backend,
- a mobile app connects to server,
- or a Machine Learning model receives data,

HTTP methods are being used.
This lesson teaches:

- What HTTP methods are
- Why they exist
- How FastAPI uses them
- Real project examples

---

1. Introduction

The goal of this lesson is to understand:

- Different HTTP methods
- Their purpose
- How to create them in FastAPI

Think of HTTP methods as:
Actions that a client wants to perform on a server

---

2. Project Overview

Before learning methods individually, it's important to understand the structure of a typical API project.

Example Project: Student Management System
Imagine you are building an API for managing students.
You may want to:

- Add a student
- View student data
- Update student info
- Delete student record

Each action uses a different HTTP method.

---

3. HTTP Methods

This is the core concept.

What is HTTP?

HTTP stands for:

HyperText Transfer Protocol

It is the communication system used between:

- Browser ↔ Server
- App ↔ Backend
- Frontend ↔ API

What are HTTP Methods?

HTTP methods tell the server:

What action should be performed

Main HTTP Methods

The most important ones are:

Method	Purpose
GET	Retrieve data
POST	Send/Create data
PUT	Update entire data
PATCH	Update partial data
DELETE	Remove data

---

1 GET Method

Purpose:

Used to **retrieve/fetch data**

Example:

Get all students:

/students

FastAPI Example:

```
@app.get("/students")
def get_students():
  return {"students": ["Ali", "Ahmed"]}
```

What happens?

When user visits:

```
http://127.0.0.1:8000/students
Server returns:
{
  "students": ["Ali", "Ahmed"]
}
```

---

2 POST Method

Purpose:

Used to **send/create new data**

Example:

Add a new student

FastAPI Example:

```
@app.post("/students")
def add_student():
   return {"message": "Student added"}
```

Important Idea:

POST usually sends:

- Form data
- JSON data
- User input

Real Example:

- Signup forms
- Login forms
- Uploading data

---

3 PUT Method

Purpose:

Used to **update entire existing data**

Example:

Replace all student information.

FastAPI Example:

```
@app.put("/students/{id}")
def update_student(id: int):
    return {"message": f"Student {id} updated"}
```

---

4 PATCH Method

Purpose:

Used for **partial update**

Difference Between PUT & PATCH

PUT	PATCH
Updates complete data	Updates only specific fields

Example:

Only update student email.

---

5 DELETE Method

Purpose:

Used to remove data

FastAPI Example:

```
@app.delete("/students/{id}")
def delete_student(id: int):
    return {"message": f"Student {id} deleted"}
```

Easy Real-World Analogy

Imagine a notebook of student records.

Action	HTTP Method
Read notebook	GET
Add new page	POST
Rewrite full page	PUT
Edit one line	PATCH
Remove page	DELETE

Client-Server Flow

Example:

User wants student data.

Step-by-step:

1. Browser sends GET request
2. FastAPI receives request
3. Function executes
4. Response returns

⚡ **FastAPI Syntax Pattern**

All HTTP methods follow similar structure:

```
@app.method("/route")
def function():
```

Examples:

```
@app.get("/")
@app.post("/")
@app.put("/")
@app.delete("/")
```

🧪 ⏱ **17:48 – HTTP Methods Demo**

This section usually demonstrates:

- Running API
- Testing methods
- Viewing responses

📄 **Automatic Swagger Docs**

FastAPI automatically provides testing interface.

Open:

http://127.0.0.1:8000/docs

You can:

- Test GET
- Send POST requests
- Delete data
- View responses

Without writing frontend code.

🧠 **Why this is powerful**

Normally API testing requires:

- Postman

- Frontend app

FastAPI gives:

✅ Built-in testing UI

🛠️ ⏱️ 20:21 – Project with Code Demo

Now methods are combined into a mini project.

📑 Example Full CRUD API

CRUD means:

Letter	Meaning
C	Create
R	Read
U	Update
D	Delete

📜 Example FastAPI CRUD

```
from fastapi import FastAPI

app = FastAPI()

students = []

# CREATE
@app.post("/students")
def add_student(name: str):
    students.append(name)
    return {"message": "Student added"}

# READ
@app.get("/students")
def get_students():
    return students

# UPDATE
@app.put("/students/{index}")
def update_student(index: int, name: str):
```

```
    students[index] = name
    return {"message": "Updated"}

# DELETE
@app.delete("/students/{index}")
def delete_student(index: int):
    students.pop(index)
    return {"message": "Deleted"}
```

🔍 **Understanding the Flow**

POST

Adds new data.

GET

Reads all data.

PUT

Changes existing data.

DELETE

Removes data.

🧠 **Important Missing Concepts (Very Necessary)**

📌 **What is an Endpoint?**

Endpoint = API URL

Example:

/students

This is where requests are sent.

📌 **What is JSON?**

Most APIs communicate using JSON.

Example:

```
{
  "name": "Ali",
  "age": 22
}
```

JSON is lightweight and easy to read.

📌 Path Parameters

Example:

/students/{id}

If user requests:

/students/5

Then:

id = 5

📌 Status Codes (Very Important)

Server also sends status codes.

Code	Meaning
200	Success
201	Created
404	Not found
500	Server error

📌 Why APIs Matter in ML

Machine Learning models use APIs heavily.

Example:

- User uploads image
- API sends image to ML model
- Model predicts result
- API returns prediction

🧠 Final Summary

HTTP methods define actions:

Method	Action
GET	Read
POST	Create
PUT	Full Update
PATCH	Partial Update
DELETE	Remove

FastAPI makes this easy because:

- Simple syntax
- Automatic docs
- Easy testing
- Fast development

🎯 Memory Trick

CRUD Mapping

CRUD	HTTP
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

🚀 What You Should Learn Next

After this topic, the best next concepts are:

1. Path Parameters
2. Query Parameters
3. Request Body
4. Pydantic Models
5. Database Integration
6. ML Model Deployment with FastAPI

These are the real building blocks of professional APIs.